

Tutorial Básico de GitHub no Laboratório

Professor: Luciano Rocha

Salvando seus códigos no GitHub usando Windows

Objetivo

Neste tutorial, você aprenderá a utilizar **Git e GitHub em um computador Windows do laboratório** para salvar os códigos desenvolvidos durante as aulas.

Os computadores são compartilhados. Por isso, além de aprender os comandos básicos, é importante tomar alguns cuidados para **não deixar sua conta conectada no computador**.

Ao final, você saberá:

- criar um repositório no GitHub;
- instalar/verificar o Git;
- configurar seu nome e e-mail;
- baixar seu repositório;
- salvar os códigos das aulas;
- enviar alterações para o GitHub;
- atualizar seus códigos;
- utilizar sua conta com segurança em um computador compartilhado.

1. Git e GitHub

Apesar dos nomes parecidos, são ferramentas diferentes.

Git é o programa que controla as versões dos arquivos do projeto.

GitHub é uma plataforma online onde podemos armazenar nossos repositórios e códigos.

O processo será:

COMPUTADOR DO LABORATÓRIO

↓

Git

↓

GitHub

Você programa no computador e utiliza o Git para registrar e enviar seus códigos ao GitHub.

2. Criando sua conta no GitHub

Acesse:

<https://github.com>

Clique em **Sign up** e faça seu cadastro.

Cada aluno deverá utilizar **sua própria conta**.

Não compartilhe:

- sua senha;
- códigos de autenticação;
- tokens;
- acesso à sua conta.

Importante

Como estamos utilizando computadores compartilhados, **não deixe sua conta conectada ao terminar a aula**.

3. Verificando o Git no Windows

Nos computadores do laboratório, o Git poderá já estar instalado.

Abra o menu **Iniciar** do Windows e procure:

```
Git Bash
```

Abra o programa e execute:

```
git --version
```

Se aparecer algo semelhante a:

```
git version 2.xx.x.windows.x
```

o Git está instalado corretamente.

Caso o Git não esteja instalado, informe ao professor.

4. Criando seu repositório

Entre no GitHub.

Clique em:

New repository

No campo **Repository name**, coloque:

aulas-programacao

Selecione:

Public

Marque:

Add a README file

Depois clique em:

Create repository

Pronto. Este será o local onde seus códigos das aulas serão armazenados.

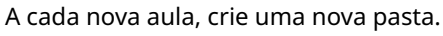
5. Organização do repositório

Não é necessário criar um repositório para cada aula.

Utilizaremos pastas.

Exemplo:

```
aulas-programacao
|
├─ aula01
|   ├── exercicio1.js
|   └── exercicio2.js
|
├─ aula02
|   ├── exemplo.js
|   └── atividade.js
|
└─ aula03
```



Abra o **Explorador de Arquivos do Windows**.

Documentos

Projetos

Documentos
└─ Projetos

No GitHub, abra seu repositório:

Clique em:

Copie o endereço.

<https://github.com/seuusuario/aulas-programacao.git>

Agora abra:

```
Documentos → Projetos
```

Clique com o botão direito e escolha:

Git Bash Here

Execute:

```
git clone https://github.com/seuusuario/aulas-programacao.git
```

Depois entre na pasta:

```
cd aulas-programacao
```

O repositório do GitHub agora está no computador.

```
GITHUB
  ↓
git clone
  ↓
COMPUTADOR
```

8. Configurando seu nome e e-mail

Atenção: estamos em um laboratório

Não utilizaremos `--global`, pois o computador é compartilhado por vários alunos.

Entre primeiro na pasta do seu repositório:

```
cd aulas-programacao
```

Configure seu nome:

```
git config user.name "Seu Nome"
```

Configure o e-mail utilizado no GitHub:

```
git config user.email "seuemail@email.com"
```

Exemplo:

```
git config user.name "Joao Silva"  
git config user.email "joao@email.com"
```

Essa configuração será aplicada **somente ao repositório atual**.

9. Conferindo sua identificação

Antes de começar, confira:

```
git config user.name
```

Depois:

```
git config user.email
```

Verifique se aparecem **seu nome e seu e-mail**.

Isso é especialmente importante em computadores compartilhados.

Atenção

`user.name` e `user.email` identificam quem realizou o commit.

Eles **não são sua senha do GitHub**.

10. Abrindo o projeto no VS Code

Dentro da pasta do projeto, execute:

```
code .
```

Se o comando não funcionar:

1. Abra o **Visual Studio Code**.
2. Clique em **File → Open Folder**.
3. Abra:

```
Documentos
→ Projetos
→ aulas-programacao
```

11. Criando a pasta da aula

No VS Code, crie:

```
aula01
```

Dentro dela, coloque os códigos desenvolvidos durante a aula.

Exemplo:

```
aulas-programacao
├──
└── aula01
    ├── exercicio1.js
    └── exercicio2.js
```

12. Criando um código

Exemplo:

```
let nome = "Maria";

console.log("Olá " + nome);
```

Salve o arquivo:

```
CTRL + S
```

13. Abrindo o terminal no VS Code

No VS Code:

Terminal → New Terminal

O terminal será aberto na parte inferior.

Verifique se está na pasta:

```
aulas-programacao
```

14. Verificando as alterações

Execute:

```
git status
```

O Git mostrará os arquivos que foram criados ou modificados.

Use esse comando sempre que quiser saber:

"O que mudou no meu projeto?"

15. Preparando os arquivos

Execute:

```
git add .
```

O ponto indica que queremos adicionar as alterações existentes no projeto.

Depois você pode conferir:

```
git status
```

16. Criando um commit

Agora registre as alterações:

```
git commit -m "Adiciona exercícios da aula 1"
```

O **commit** registra uma versão do seu projeto.

Use mensagens que expliquem o que foi realizado.

Exemplos:

```
git commit -m "Adiciona atividade de JavaScript"
```

```
git commit -m "Adiciona exercícios de funções"
```

```
git commit -m "Corrige exercício da aula 3"
```

Evite mensagens como:

```
git commit -m "coisa"
```

ou:

```
git commit -m "teste"
```

17. Enviando seus códigos

Depois do commit:

```
git push
```

O GitHub poderá solicitar autenticação.

Siga a tela apresentada pelo Git/GitHub e utilize **somente sua própria conta**.

Importante

O GitHub não utiliza mais a senha comum da conta como senha do comando Git via HTTPS.

Dependendo da configuração do computador, a autenticação poderá ser realizada pelo navegador ou pelo sistema de autenticação do Git.

18. Não salvar suas credenciais

Estamos utilizando um **computador compartilhado**.

Caso apareça alguma opção semelhante a:

Remember me

Remember credentials

ou opção para manter sua conta conectada:

NÃO MARQUE.

Não deixe:

- senha salva;
- token salvo;
- conta conectada;
- navegador conectado ao GitHub.

19. Confira se o envio funcionou

Depois do:

```
git push
```

Abra seu repositório no site do GitHub.

Atualize a página.

Confira:

- se a pasta da aula apareceu;
- se os arquivos estão disponíveis;
- se o último commit aparece no repositório.

Somente considere a atividade salva depois de confirmar que ela está no GitHub.

20. Rotina para salvar seus códigos

Depois que tudo estiver configurado, este será o processo mais utilizado.

Primeiro:

```
git status
```

Depois:

```
git add .
```

Registre:

```
git commit -m "Atividades da aula"
```

Envie:

```
git push
```

Memorize:

CRIAR/ALTERAR CÓDIGO

↓

CTRL + S

↓

git status

↓

git add .

↓

git commit

↓

git push

↓

AUTENTICAR NO GITHUB

↓

CONFERIR NO GITHUB

21. Nas próximas aulas

Você não precisa criar outro repositório.

Se o seu repositório ainda estiver no computador, abra:

```
Documentos
→ Projetos
→ aulas-programacao
```

Abra essa pasta no VS Code.

Antes de programar, execute:

```
git pull
```

Isso verifica e baixa as alterações que estiverem no GitHub.

Depois crie a pasta da nova aula:

```
aula02
```

Faça seus códigos normalmente.

22. Se estiver usando outro computador do laboratório

Caso seu repositório **não exista naquele computador**, abra a pasta `Projetos` e faça novamente:

```
git clone URL_DO_SEU_REPOSITORIO
```

Exemplo:

```
git clone https://github.com/seuusuario/aulas-programacao.git
```

Entre na pasta:

```
cd aulas-programacao
```

Configure sua identificação para aquele repositório:

```
git config user.name "Seu Nome"
git config user.email "seuemail@email.com"
```

Depois:

```
code .
```

Agora você poderá continuar trabalhando.

23. Se o repositório já estiver no computador

Não faça outro `git clone`.

Abra o repositório existente e execute:

```
git pull
```

Podemos lembrar assim:

```

      git push
COMPUTADOR  ----->  GITHUB
      git pull
COMPUTADOR <-----  GITHUB
```

24. Começou a aula?

Se o repositório já estiver no computador:

1. Abra o projeto.

2. Confira sua identificação:

```
git config user.name
git config user.email
```

3. Atualize:

```
git pull
```

4. Comece a programar.

25. Terminou a aula?

Salve seus arquivos:

```
CTRL + S
```

Depois:

```
git status
git add .
git commit -m "Atividades da aula"
git push
```

Autentique-se quando solicitado.

Depois:

1. Abra o GitHub.
2. Confira se os arquivos foram enviados.
3. Faça **logout da sua conta do GitHub**.
4. Feche o navegador.
5. Feche o VS Code.

26. Cuidados importantes no laboratório

Nunca faça:

- salvar sua senha no computador;
- deixar sua conta GitHub conectada;
- compartilhar sua senha;
- utilizar a conta de outro aluno;
- colocar senha ou token dentro dos códigos;
- colocar senha ou token dentro do endereço do repositório;
- criar uma chave SSH pessoal e deixá-la no computador do laboratório.

Sempre faça:

- utilize sua própria conta;
 - confira seu nome e e-mail no Git;
 - autentique-se somente quando necessário;
 - confira se o `push` funcionou;
 - faça logout ao terminar.
-

27. Comandos principais

Comando	Para que serve
<code>git --version</code>	Verifica se o Git está instalado
<code>git clone URL</code>	Baixa seu repositório
<code>git config user.name "Nome"</code>	Define o nome naquele repositório
<code>git config user.email "email"</code>	Define o e-mail naquele repositório
<code>git pull</code>	Baixa atualizações
<code>git status</code>	Mostra arquivos modificados
<code>git add .</code>	Prepara as alterações
<code>git commit -m "mensagem"</code>	Registra uma versão
<code>git push</code>	Envia os códigos para o GitHub

28. Resumo — Primeiro acesso no computador

Caso seu projeto ainda não esteja naquele computador:

```
git clone URL_DO_REPOSITORIO
cd nome-do-repositorio

git config user.name "Seu Nome"
git config user.email "seuemail@email.com"

code .
```

29. Resumo — Início da aula

Confira sua identificação:

```
git config user.name
git config user.email
```

Atualize seu projeto:

```
git pull
```

Depois comece a programar.

30. Resumo — Final da aula

```
git status
git add .
git commit -m "Atividades da aula"
git push
```

Depois:

```
1. Autenticar
   ↓
2. Conferir o GitHub
   ↓
3. Fazer logout
   ↓
4. Fechar o navegador
```

Atividade Prática

- [] Entre na sua conta do GitHub.
 - [] Crie o repositório `aulas-programacao`.
 - [] Abra a pasta `Projetos` no computador.
 - [] Clone seu repositório.
 - [] Entre na pasta clonada.
 - [] Configure seu nome e e-mail apenas no repositório.
 - [] Confira seu nome e e-mail.
 - [] Abra o projeto no VS Code.
 - [] Crie a pasta `aula01`.
 - [] Crie pelo menos um arquivo de código.
 - [] Salve o arquivo.
 - [] Execute `git status`.
 - [] Execute `git add .`.
 - [] Faça um commit.
 - [] Execute `git push`.
 - [] Faça a autenticação quando solicitada.
 - [] Confira os arquivos no site do GitHub.
 - [] Faça logout da sua conta.
-

Para memorizar

Começou a aula

```
git config user.name  
git config user.email  
git pull
```

Terminou a aula

```
git status  
git add .  
git commit -m "Atividades da aula"  
git push
```

E antes de sair

Confira o GitHub → faça logout → feche o navegador.

Regra principal

A atividade só está realmente salva quando você confirma que seus arquivos foram enviados para o seu repositório no GitHub.